

Hadoop & MapReduce: Distributed Processing for Unstructured Data

BDA POP — Session 09/15/2026

Why this session exists

A platform the size of Zomato or Swiggy doesn't just store structured rows like cost and rating — it accumulates unstructured text at the same time: millions of customer reviews and order notes a day, in free-form language, with no fixed schema. A single machine running a script has two hard limits: how much data fits in its RAM, and how fast one CPU can walk through it. Text at real platform volume routinely blows past the first limit long before the second one matters. Hadoop exists to remove that ceiling — spread the data itself across many machines' disks, then run the computation on each machine where its own slice of data already lives, instead of hauling everything to one place first. MapReduce is the programming model that makes “run the same computation on many independent slices, then combine the partial results” a repeatable pattern instead of custom plumbing every time. The classic example is counting words across a huge document set — the same shape of problem as tallying sentiment words across a huge pile of reviews, which is what we'll build below.

STEP 0: LIBRARIES

```
# `parallel` ships with base R -- no install.packages() needed on posit.cloud.  
library(parallel)
```

STEP 1: THE RUNNING DATASET, EXTENDED WITH UNSTRUCTURED REVIEW TEXT

Same self-generated Zomato/Swiggy restaurant dataset as previous sessions — built with `set.seed(42)` so it runs identically everywhere, no external files required. This week adds a `review_text` column: unstructured, free-form text with no fixed schema, exactly the kind of data MapReduce was built for.

```
set.seed(42)  
n_restaurants <- 200  
  
restaurant_data <- data.frame(  
  restaurant_id = 1:n_restaurants,  
  platform = sample(c("Zomato", "Swiggy"), n_restaurants, replace = TRUE),  
  average_cost_two = round(runif(n_restaurants, min = 150, max = 2500)),  
  distance_center_km = round(runif(n_restaurants, min = 0.5, max = 15), 1),  
  rating = round(runif(n_restaurants, min = 1.5, max = 4.9), 1),  
  votes = round(runif(n_restaurants, min = 10, max = 1500))  
)
```

```

# More votes + higher cost => higher estimated revenue (linear relationship)
restaurant_data$estimated_revenue <- round(
  (restaurant_data$votes * 1.5) + (restaurant_data$average_cost_two * 0.4) -
  (restaurant_data$distance_center_km * 50) + rnorm(n_restaurants, mean = 500, sd =
100)
)
restaurant_data$estimated_revenue[restaurant_data$estimated_revenue < 100] <- 100

# Expensive restaurants are more likely to offer online delivery (binary relationshi
p)
prob_delivery <- 1 / (1 + exp(-(-1 + 0.0015 * restaurant_data$average_cost_two)))
restaurant_data$has_online_delivery <- rbinom(n_restaurants, 1, prob_delivery)

# Price tier from cost (non-linear rule relationship)
restaurant_data$price_range <- factor(cut(restaurant_data$average_cost_two,
breaks = c(0, 400, 800, 1500, Inf),
labels = c("Budget", "Moderate", "Premium", "Luxur
y"))))

# --- EXTENSION: unstructured review text, one free-text blurb per restaurant ---
# A small fixed vocabulary of positive/negative/neutral review words. Higher-rated
# restaurants draw positive words more often -- so review sentiment tracks the
# rating column, the same way real review text tracks real satisfaction.
positive_words <- c("delicious", "fast", "friendly", "fresh", "amazing", "great",
  "tasty", "quick", "clean", "recommend")
negative_words <- c("cold", "late", "rude", "stale", "slow", "disappointing",
  "soggy", "overpriced", "bland", "missing")
neutral_words <- c("order", "delivery", "packaging", "portion", "menu", "app",
  "price", "restaurant", "food", "service")
vocab <- c(positive_words, negative_words, neutral_words)

words_per_review <- 40
review_text <- character(n_restaurants)
for (i in 1:n_restaurants) {
  # rating runs 1.5-4.9; rescale to a 0-1 weight on positive words
  pos_weight <- (restaurant_data$rating[i] - 1) / 4
  word_probs <- c(rep(pos_weight, length(positive_words)),
    rep(1 - pos_weight, length(negative_words)),
    rep(0.6, length(neutral_words)))
  review_text[i] <- paste(sample(vocab, words_per_review, replace = TRUE, prob = word
_probs),
    collapse = " ")
}
restaurant_data$review_text <- review_text

cat("Sample review (restaurant 1, rating", restaurant_data$rating[1], "):\n")

```

```

## Sample review (restaurant 1, rating 3.2 ):

```

```
cat(restaurant_data$review_text[1], "\n")
```

```
## stale quick stale food clean fast friendly disappointing service delicious tasty t  
asty app disappointing slow rude bland quick fast soggy food soggy soggy service cold  
stale clean app portion app packaging delivery great portion rude rude food delivery  
disappointing bland
```

What MapReduce actually does

MapReduce breaks a job into three phases:

1. **Map** — split the input into pieces and run the same function on each piece independently, producing key/value pairs (here: a word, and a count of 1 for each time it appears).
2. **Shuffle & sort** — group every pair by key across all the pieces, so all the counts for the same word end up together, regardless of which piece they came from.
3. **Reduce** — combine the grouped values into a final answer per key (here: sum the 1s into a total count per word).

The map step never needs to see the whole dataset — only its own piece — which is exactly what lets it run on many machines at once.

That independence pays off a second way: because map and reduce functions are deterministic — the same input block always produces the same output — a task that fails or a machine that dies mid-job can simply be re-run somewhere else. Ten mappers running, two crash: the framework reschedules those two, the other eight keep going, and the final answer is unaffected. Failure is cheap to recover from precisely because the functions are pure.

Hadoop's architecture underneath MapReduce

MapReduce is the computation model; Hadoop is the system that runs it at cluster scale, built from two other pieces:

- **HDFS (Hadoop Distributed File System)** splits a file into fixed-size blocks — 128 MB by default, deliberately large — and spreads them across the disks of many machines (DataNodes). A NameNode holds only metadata (which blocks make up which file, and where each lives); it never stores file data itself. Fewer, bigger blocks means far less metadata to track, and HDFS is optimized for streaming large files end-to-end, not seeking to small random offsets (that's HBase's job, below). Each block is replicated three times by default, spread across different DataNodes and — where the cluster spans racks — different racks, so losing a whole rack still leaves two copies standing. DataNodes heartbeat to the NameNode; a missed heartbeat means the NameNode schedules fresh replicas onto healthy nodes automatically. Honest caveat: the NameNode itself is a single point of failure in this design — production clusters run NameNode high-availability pairs specifically because of it.
- **YARN (Yet Another Resource Negotiator)** decides which machine runs which task and how much CPU/memory it gets, split across three roles: one **ResourceManager** per cluster tracks all available capacity and hands out containers; one **NodeManager** per machine runs and monitors the containers

assigned to it; one **ApplicationMaster** per submitted job negotiates that job's containers and supervises only that job's own tasks, restarting the ones that fail. Hadoop 1.0's single JobTracker did all of this at once and hit scaling limits; Hadoop 2.0 split it into these three roles specifically to fix that. Because YARN isn't wired to MapReduce specifically, Spark, Hive, and other engines can all run on the same cluster, sharing the same capacity.

The payoff of putting HDFS and YARN together: when a MapReduce job runs, the ResourceManager schedules each map task's container on (or near) the DataNode that already holds that task's input block — the computation moves to the data, not the other way around. That's the part a single-machine script structurally cannot do, no matter how it's written.

The ecosystem built on top: Hive, Pig, and HBase

Raw MapReduce is what makes distributed computation possible, but almost nobody at Zomato-or-Swiggy scale writes raw Map and Reduce functions for everyday work. Three tools sit on top of HDFS+YARN and cover the jobs a hand-written MapReduce job would be painfully slow to build:

- **Hive** gives analysts a SQL-like query language (HiveQL) over data that's already sitting in HDFS. Behind the scenes, Hive compiles each query down into a MapReduce (or newer Tez) job — the analyst never writes a Map or Reduce function. Without it, “how many one-star reviews came from restaurants in Bangalore last month?” is a ticket to an engineer, not a query someone runs themselves before lunch.
- **Pig** gives data engineers a scripting language (Pig Latin) for describing a pipeline — load, filter, group, join — as a sequence of steps, also compiled down to MapReduce. It's the tool for the daily job that turns yesterday's raw, messy review and order logs into the cleaned, structured table Hive queries run against. Without it, every format change in the raw logs means hand-editing bespoke MapReduce code instead of one pipeline script.
- **HBase** solves a problem HDFS+MapReduce structurally can't: getting one record back instantly. HDFS is built for scanning huge files in bulk; a MapReduce job to find “this one customer's most recent order” would mean scanning blocks across the whole cluster for a single row. HBase is a NoSQL, column-oriented store (modeled on Google's Bigtable) that sits on top of HDFS and answers exactly that kind of point lookup in milliseconds — the difference between a support agent pulling up an order live on a call versus reading from a report that's hours old.

```

# Reference data, not computed from the dataset -- this just displays the comparison
the same
# way we display everything else in this document. None of these tools run inside R;
each one
# runs on top of a real HDFS+YARN cluster.
ecosystem_tools <- data.frame(
  Tool = c("Hive", "Pig", "HBase"),
  Provides = c("SQL-like queries (HiveQL) on data sitting in HDFS",
               "A scripting language (Pig Latin) for multi-step ETL pipelines",
               "Real-time, random read/write access to individual rows (NoSQL)"),
  Runs_As = c("Compiled down to MapReduce/Tez jobs",
              "Compiled down to MapReduce jobs",
              "Its own service layer on top of HDFS -- no MapReduce per lookup"),
  Reached_For_When = c("An analyst needs an answer today without writing MapReduce co
de",
                      "An engineer needs a repeatable daily transform pipeline, not
one-off code",
                      "A support agent needs one customer's record back in milliseco
nds")
)
print(ecosystem_tools, row.names = FALSE)

```

```

##      Tool                                          Provides
##      Hive          SQL-like queries (HiveQL) on data sitting in HDFS
##      Pig    A scripting language (Pig Latin) for multi-step ETL pipelines
##      HBase Real-time, random read/write access to individual rows (NoSQL)
##                                          Runs_As
##                                          Compiled down to MapReduce/Tez jobs
##                                          Compiled down to MapReduce jobs
##      Its own service layer on top of HDFS -- no MapReduce per lookup
##                                          Reached_For_When
##      An analyst needs an answer today without writing MapReduce code
##      An engineer needs a repeatable daily transform pipeline, not one-off code
##      A support agent needs one customer's record back in milliseconds

```

Hive and Pig both still compile down to the same batch MapReduce model described above; HBase is the one piece of this ecosystem that steps outside it entirely, because “wait for a batch job” isn’t good enough for a live lookup.

The naive approach: everything in one machine’s memory

The most direct way to count words is a single loop over every review, keeping one running tally. It’s correct, and simple to read:

```
naive_wordcount <- function(reviews) {
  tally <- new.env()
  for (review in reviews) {
    words <- strsplit(review, " ")[[1]]
    for (w in words) {
      if (is.null(tally[[w]])) tally[[w]] <- 0L
      tally[[w]] <- tally[[w]] + 1L
    }
  }
  sort(unlist(as.list(tally)), decreasing = TRUE)
}

t_naive <- system.time(naive_result <- naive_wordcount(restaurant_data$review_text))
cat("Elapsed on", n_restaurants, "reviews:", t_naive["elapsed"], "seconds\n")
```

```
## Elapsed on 200 reviews: 0.012 seconds
```

```
head(naive_result, 5)
```

```
## restaurant      food      portion      price      menu
##           323         305         305         304         303
```

At 200 restaurants this finishes instantly — the flaw isn't speed, it's the architecture. This function requires every review to already be sitting in one process's memory before it can start. That assumption breaks long before CPU speed becomes the bottleneck:

```
avg_review_bytes <- 150           # a short real review, roughly this many bytes
reviews_per_day  <- 500e6         # stated assumption: platform-wide daily review volume
trailing_window_days <- 90        # a typical trailing analytics window
single_machine_ram_gb <- 256      # generous ceiling for one high-memory server

total_bytes <- avg_review_bytes * reviews_per_day * trailing_window_days
total_tb    <- total_bytes / 1e12
ram_needed_vs_ceiling <- (total_tb * 1000) / single_machine_ram_gb

cat("Trailing", trailing_window_days, "-day review volume: ~", round(total_tb, 2), "TB\n", sep = " ")
```

```
## Trailing90-day review volume: ~6.75TB
```

```
cat("Single-machine RAM ceiling:", single_machine_ram_gb, "GB\n")
```

```
## Single-machine RAM ceiling: 256 GB
```

```
cat("That volume is", round(ram_needed_vs_ceiling, 1), "x the single-machine ceiling\n")
```

```
## That volume is 26.4 x the single-machine ceiling
```

Metric	Value	Threshold	Status
90-day review volume vs. single-machine RAM	~26x over	$\leq 1x$ (must fit in RAM)	Flagged

Reading the panel: the naive function above isn't slow — it never even gets the chance to be. At real platform volume, the input alone is roughly 26 times larger than a single machine's memory can hold. No amount of loop optimization fixes that; the data has to live somewhere else from the start.

The fix: partition like HDFS would, map locally, reduce the small result

HDFS would already have this data pre-split into blocks spread across many DataNodes. We simulate that with `split()`, run the map step on each partition independently with `mclapply()` (a stand-in for “one mapper task per node”), then reduce the small partial tallies — never the raw text — into one final answer:

```

n_cores <- detectCores()
n_partitions <- 8
partitions <- split(restaurant_data$review_text,
                    cut(seq_along(restaurant_data$review_text), n_partitions, labels
= FALSE))

# Map: runs independently on each partition -- this is the piece that would
# execute on the node holding that partition's data in a real cluster.
map_fn <- function(reviews) {
  words <- unlist(strsplit(reviews, " "))
  table(words)
}

# Reduce: combines two partial tallies into one. Only ever moves small,
# already-aggregated counts across the network -- never the raw text.
reduce_fn <- function(a, b) {
  keys <- union(names(a), names(b))
  setNames(sapply(keys, function(k) {
    (if (k %in% names(a)) a[[k]] else 0) + (if (k %in% names(b)) b[[k]] else 0)
  }), keys)
}

t_mapreduce <- system.time({
  partial_counts <- mclapply(partitions, map_fn, mc.cores = n_cores)
  final_counts <- sort(Reduce(reduce_fn, partial_counts), decreasing = TRUE)
})

cat("Elapsed on", n_partitions, "partitions,", n_cores, "cores:", t_mapreduce[["elapsed"]], "seconds\n")

```

```
## Elapsed on 8 partitions, 8 cores: 0.057 seconds
```

```
head(final_counts, 5)
```

```
## restaurant      food      portion      price      menu
##           323         305         305         304         303
```

```

same_answer <- identical(as.numeric(naive_result[sort(names(naive_result))]),
                        as.numeric(final_counts[sort(names(final_counts))]))
cat("Partitioned map+reduce result matches the naive single-machine result:", same_answer, "\n")

```

```
## Partitioned map+reduce result matches the naive single-machine result: TRUE
```



```
cluster_nodes <- 200    # stated assumption: a modest real Hadoop cluster
shard_tb <- total_tb / cluster_nodes
shard_gb <- shard_tb * 1000
cat("Per-node shard at", cluster_nodes, "nodes: ~", round(shard_gb, 2), "GB\n", sep =
"")
```

```
## Per-node shard at200nodes: ~33.75GB
```

```
cat("Single-machine RAM ceiling:", single_machine_ram_gb, "GB\n")
```

```
## Single-machine RAM ceiling: 256 GB
```

Metric	Value	Threshold	Status
Per-node shard size (200- node cluster)	~34 GB	≤ 256 GB RAM ceiling	Within threshold
Word-count result vs. naive single-machine run	identical	must match	Within threshold

Reading the panel: splitting the same 90-day volume across 200 nodes brings each machine’s share down to roughly 34 GB — comfortably inside the RAM ceiling that the single-machine version couldn’t clear at all. The correctness check confirms this isn’t a different answer computed a faster way — it’s the exact same tally, just computed where the data already lives.

An honest complication: distribution isn’t free

Look at the two elapsed times actually measured above, on our 200-restaurant, 8,000-word toy corpus: the single-loop naive version finished in a few milliseconds; the 8-way partitioned Map+Reduce version took noticeably *longer*, not shorter. That’s real, not a mistake — forking worker processes and reassembling their results costs time, and at this small a scale that coordination overhead outweighs the work being distributed. The lesson isn’t “MapReduce is slower” — it’s that distribution only pays for itself once each machine’s local share of the work is large enough to be worth coordinating. That’s exactly the 26x-over-RAM scenario above, and exactly why nobody runs Hadoop on 8,000 words: the value shows up at the volume where a single machine can’t do the job at any speed, not as a speed trick at small scale.

Where MapReduce falls short — and why the pattern outlived the engine

MapReduce’s honest weak spot is iterative work. Every job reads its input from disk and writes its output back to disk before the next stage starts; an algorithm that needs a hundred passes over the same data pays that disk round-trip a hundred times, with no way to keep intermediate results in memory across passes. That’s a batch model built for throughput on one large pass, not for interactive queries or repeated iteration — and it’s the specific gap Spark closed by keeping data in memory across steps, which is why Spark displaced MapReduce for machine-learning workloads rather than replacing it everywhere.

What didn’t fade is the shape of the problem. A modern batch embedding pipeline — chunk millions of documents, compute an embedding vector for each, route vectors to partitions, assemble the partitions into index shards — is map, shuffle, and reduce under a different name, tuned by the same instincts: chunk size versus coordination overhead, a shuffle step that dominates cost, tasks that re-run safely on failure because the work is deterministic.

Summary

Check	Result
Naive single-machine approach vs. 90-day review volume	Flagged — ~26x over a single machine’s RAM
Partitioned Map+Reduce shard size vs. RAM ceiling (200 nodes)	Within threshold — ~34 GB per node
Partitioned result vs. naive result	Within threshold — identical word counts
Partitioned runtime vs. naive runtime, at toy (8,000-word) scale	Flagged — coordination overhead exceeds the naive runtime here

MapReduce’s win isn’t raw speed on any given machine — it’s turning a job that’s structurally impossible on one machine into a set of independent, combinable pieces. Hadoop’s HDFS+YARN pair is what makes that split automatic: data pre-sharded across disks, computation scheduled to run where each shard already sits.